The Machinery of Modal Logic

STUDENT EDITION · with Professor Logic

Must, might, can't, could-have: the logic of alternatives. Built from the same three knobs, and derived, checked, and audited every step of the way.

PROFESSOR LOGIC

Third machine. If you've been with me since the logic book, you know the drill; if you're new, you'll catch it in a page. We build reasoning machines from three knobs, **V** (what values exist), **G** (how they combine), **θ** (what counts), and we never, ever take a rule on faith. Every rule is derived or it's labelled as a choice.

Today's machine handles the most human words in your vocabulary: **must**, **might**, **can't**, **could have**, **has to**. You use them hundreds of times a day. "I *might* come to the party." "You *must* be joking." "That *can't* be right." "It *could have* gone differently." Every one of those words reaches beyond how things *are* into how things *could be*, and our machines so far cannot see that far. Today we upgrade them until they can.

THE TRUST LABELS, AS ALWAYS

FORCED Derived or checked by complete enumeration, every case, no exceptions. Stamped boards cite their check.

STIPULATED A choice. Could be otherwise. We show the alternative.

HISTORY Real people, real years, real problems.

ANALOGY A bridge to walk across, never a house to live in.

PROFESSOR LOGIC · RIGOR CHECK

One promise specific to this book, because modal logic attracts more mysticism than any subject I teach: we will talk about "possible worlds," and **at no point will you be asked to believe in parallel universes.** You'll see exactly what the "worlds" are, rows in a value table, nothing spookier, and the machinery will run whether you think they're metaphysically real, useful fictions, or game save-files. The map is not the territory. The scaffolding is not the building. Hold everything as loosely as it deserves and no more tightly.

THE JOURNEY

CHAPTER 1

Words That Bend Truth

Two true sentences that are true in completely different ways, and a machine that can't tell them apart.

PROFESSOR LOGIC

Read these two sentences. Both are true. Feel the difference:

"My phone is charging." **"If it's raining, then it's raining."**

The first one is true, but it didn't *have* to be. You could've forgotten the cable. Five minutes ago it was false. It's true *the way things happened to go*. The second one could not have failed no matter how tonight went. Rain or shine, it holds. And you already own the receipt: it's the logic book's machinery, checked value by value, true when P is 0 and true when P is 1. Its truth never once touched the weather. Philosophers say the first is *contingently* true and the second *necessarily* true. Forget the jargon; you already feel the difference. "Happens to be" versus "has to be."

THE CHALKBOARD — The distinction our machine cannot see FORCED
verified: modal_1_1

```
Feed both sentences to the logic-book machine (V = {0,1}):

  "My phone is charging."              →  value: 1
  "If it's raining, it's raining."     →  value: 1

Same value. IDENTICAL. Everything the machine can say about
these sentences, it says the same for both. The difference you
just FELT — happens-to-be vs has-to-be — is invisible in V.

This is not the machine being broken. It's the machine being
poor: one number per sentence has no room for the distinction.
(Compare the calc book's jam: the question was meaningful,
the current V just had no value for it.)
```

SIMPLIFIED FOR CLARITY

"True" comes in flavors: *happens-to-be-true* ("it's raining") and *has-to-be-true* ("if it's raining, it's raining"), plus *could-be-true* ("I might come") and *can't-be-true* ("I'm in two cities at once"). Our one-number-per-sentence machine flattens all of that into 1s and 0s. The must/might layer is real, you use it daily, and V can't see it. Time for a knob upgrade.

PROFESSOR LOGIC · RIGOR CHECK · THE EXAMPLE WE REFUSED

The traditional pick for a has-to-be truth is "2 + 2 = 4," and this book almost printed it. Then we ran the audit. Take two piles of sand; add two more piles of sand; push them together. Count the piles: **one.** Two raindrops meet on a window and make one raindrop. "That's not what addition means," you object, "you have to keep the objects separate!" Exactly. Listen to what that objection admits: keeping the boundaries separate is an *agreement we bring*, a carrier choice about where one thing ends and the next begins. Hold that agreement fixed in every situation and the 2+2=4 row is flat 1s. Drop it, and the row wobbles. So its necessity is real, but it is necessity *relative to a frame we chose*: 2+2=4 is true under specific agreements about boundary distinctions, not true the way rain is wet. The flat rows we can **guarantee** are the ones whose receipts come from the per-situation machinery itself, like "if it's raining, it's raining." Want a candidate about the world that survives harder scrutiny? Try "something is happening": find an observable situation where nothing is. But notice the argument then shifts to what "observable" holds fixed. Necessity keeps pointing back at the agreements carried across the columns. Anyone who tells you arithmetic floats in a perfect heaven has mistaken a very good carrier for the territory.

The oldest version of this problem: tomorrow's sea battle HISTORY

~350 BC. **Aristotle** spotted the trap first. Consider: "*There will be a sea battle tomorrow.*" Is that true right now, or false right now? If it's already true, the battle seems locked in, the admirals' meeting tonight is theatre. If already false, equally locked. Either way, the future is fixed and deliberation is a joke. Aristotle refused both options, and the puzzle of *future contingents* has run for 2,300 years. Now, a series callback: in the logic book you met **Jan Łukasiewicz**, who attacked *this exact puzzle* in 1918 with a third *value*, turning Knob 1 to add a "not yet settled" option. Modal logic is the *other* engineering answer to the same puzzle: instead of new values, **new situations**, tomorrow-with-a-battle and tomorrow-without both held in view, neither privileged yet. One puzzle, two knob-turns. By Chapter 3 you'll be able to say precisely how they differ.

TRY IT YOURSELF, CHAPTER 1

Only uses: your ear for must/might, and the observation that {0,1} flattens it.

1. Sort these into *has-to-be*, *happens-to-be*, *could-be*, *can't-be*: (a) "A triangle has three sides." (b) "My bus was late today." (c) "I could get an A this term." (d) "This single coin landed both heads and tails."
2. Write one sentence of your own for each of the four flavors. (Group-chat material is allowed and encouraged.)
3. The flattening, hands-on: assign {0,1} values to your (a) and (b) from question 1. Now explain, in one sentence, what information the two identical values just destroyed.

ANSWERS

1. (a) has-to-be, with the necessity honestly located: the row stays flat because the word-agreement (what "triangle" means) rides fixed into every situation. The necessity lives in the definition we carry, not in a heaven of triangles; (b) happens-to-be (could've been on time); (c) could-be (open possibility); (d) can't-be, and notice its shape: "landed heads and not-heads" is the logic book's contradiction, flat 0s by machinery, not by coin design.

2. Yours, but check each against the test "could it have gone otherwise?" If yes: happens-to-be or could-be. If no: has-to-be or can't-be.

3. Both get 1, and the values no longer record that (b) could have been 0 while (a) never could. The machine kept *whether* they're true and threw away *how* they're true. That thrown-away "how" is what this book rebuilds.

CHAPTER 2

The V Upgrade: One Value Becomes a Profile

The fix isn't a new number. It's a new shape of value.

PROFESSOR LOGIC

Here's the engineering insight the whole subject stands on. In the logic book, when {0,1} couldn't handle "not yet settled," we added a value *between* 0 and 1. That was the right fix for *that* jam. It is the wrong fix for this one, "has to be true" isn't *half* true, it's true in a stronger *way*. The must/might layer isn't about degrees. It's about **alternatives**.

So here's the upgrade: a sentence's value stops being one number and becomes a **row of numbers, one per situation.** "My phone is charging": true in the situation where you plugged it in, false in the one where you forgot. "2+2=4": true in *every* situation on the row. Look, the difference from Chapter 1 just became *visible*:

THE CHALKBOARD — Value profiles STIPULATED — the upgraded V

```
Situations: s1 = tonight as it went,  s2 = you forgot the cable,
            s3 = power outage

                          s1   s2   s3
  "Phone is charging"      1    0    0     ← true HERE, not everywhere
  "If it rains, it rains"  1    1    1     ← true in EVERY situation
  "It rains AND it
     doesn't rain"         0    0    0     ← true in NO situation

(Both flat rows carry machinery receipts. Chapter 1's refused
example is the story of why that guarantee matters.)

The Chapter-1 distinction, now VISIBLE IN V:
  happens-to-be = 1 in this row's actual column, 0 somewhere else
  has-to-be     = 1 straight across
  can't-be      = 0 straight across
A "situation" is a COLUMN. That's all a "possible world" is:
a column in this table. No parallel universes were purchased.
```

SIMPLIFIED FOR CLARITY

Knob 1, the value space V, upgraded: a value = a **profile across situations**, not a single number. Think game save-files: "I've beaten the water temple" is true on one save, false on another; "the game contains a water temple" is true on every save. Same sentence, different columns. Must/might live in the *shape of the row*.

PROFESSOR LOGIC · RIGOR CHECK

Catch the label on that board: STIPULATED. Upgrading V to profiles is a *choice*, the alternative (Łukasiewicz's third value) exists, you've met it, and it solves a neighboring problem. Neither is "the truth about truth." They're different tools: degrees-of-settledness versus truth-across-alternatives. An engineer who can name the alternative to their own design has actually made a design decision; one who can't has made an assumption. Also note what we have *not* yet done: nothing so far says which situations *matter* from where. That's the missing ingredient, it's the powerful one, and it's next.

TRY IT YOURSELF, CHAPTER 2

Only uses: value profiles over given situations. No operators yet; they don't exist until Chapter 4.

1. Situations: s1 = your life today, s2 = same but you never joined your main group chat, s3 = same but school is fully online. Write the profile row (1s and 0s) for: (a) "I own a phone." (b) "I text my best friend daily." (c) "I attend class in a building."
2. Using the situations from question 1, invent one sentence that's true straight across all three columns, and one that's false straight across.
3. A friend looks at the chalkboard and says "so modal logic believes in parallel universes." Correct them in one sentence using only what a "situation" actually is in the machinery.

ANSWERS

1. Likely: (a) 1 1 1 (nothing in s2/s3 removes your phone); (b) maybe 1 0 1, false where the chat never existed (your call, defend it); (c) 1 1 0, false in the online situation. The point is that different sentences have different *shapes* across the same situations.

2. All-1s: "I am a student" works for *these three* columns, since none of them un-enrolls you; guaranteed-flat by machinery: "If I'm online, I'm online." All-0s: "I'm online and I'm not online." Note the two grades of flatness: content can happen to be flat across the columns you chose, but only machinery-receipted rows are flat across any columns whatsoever. Chapter 1's refused example lives in the gap between those grades.

3. "A situation is a column in a value table, the machinery runs identically whether you read the columns as real universes, thought experiments, or save files, so it commits you to none of them." (Believing in the universes is a philosophy add-on, sold separately, never required by the math.)

CHAPTER 3

Reach: Which Situations Count

The arrows that do all the work in this entire subject.

PROFESSOR LOGIC

Profiles gave us columns. Now the crucial question the table can't answer by itself: standing in one situation, **which other situations count as live alternatives?**

Because they don't all count, do they? From tonight, "tomorrow I ace the quiz" is a live alternative. "Tomorrow I'm the King of France" is... a column in somebody's table, sure, but not one *reachable* from here. From this save file, some game states are reachable and some aren't. From this city, some cities have direct flights. Every modal word you own, must, might, can't, quietly leans on a map of **which alternatives are in play from where.**

So we add the second ingredient: **reach**, arrows between situations. "s1 reaches s2" means: standing at s1, s2 counts as a live alternative. Draw the arrows and you've drawn the physics of your must and might.

THE CHALKBOARD — A reach map STIPULATED — you choose the arrows

```
Situations: s1 = tonight (quiz tomorrow), s2 = tomorrow, you studied,
            s3 = tomorrow, you winged it,  s4 = tomorrow, school vanished

Reach arrows:  s1 → s2      s1 → s3      s2 → s2      s3 → s2
               (s4 gets NO incoming arrow: not a live alternative
                from anywhere — and no outgoing arrows either)

Read it like a subway map:
  from s1 you can "get to" s2 or s3      (two live tomorrows)
  from s3 you can get to s2              (winged it? study next time)
  from s4 you can get nowhere at all     (remember this loner —
                                          it does something weird
                                          in Chapter 4)
WHICH arrows exist is a modelling CHOICE — that's the label.
What the arrows then FORCE is not. Sound familiar? It should:
that's been the shape of every machine in this series.
```

SIMPLIFIED FOR CLARITY

Reach = arrows saying which situations are live alternatives *from* which. Flights between cities. Moves between game states. Tomorrows available from tonight. You choose the arrows to fit your job (that's the STIPULATED part); everything after that will be forced (that's the next three chapters).

Naming rights: Leibniz's worlds, Kripke's arrows HISTORY

1710. The phrase "possible worlds" is **Leibniz**, yes, the calculus co-inventor from the last book; the man got around. He pictured God surveying all possible worlds and actualizing the best one (his infamous "best of all possible worlds"). For 250 years, possible worlds stayed philosophy, evocative, unmechanized.
1959–63. Then a teenager from Omaha, Nebraska mechanized them. **Saul Kripke** published a completeness theorem for modal logic in a top journal at nineteen, work begun while he was still in high school, and in 1963 laid out the full framework you are learning right now: situations, plus a *relation* saying which reach which. The arrows were the missing part. Everyone had worlds; Kripke added the reach map, and suddenly the whole subject computed. It's called *Kripke semantics*, and when you meet that name in a textbook, you'll know it's the subway map you drew today.

TRY IT YOURSELF, CHAPTER 3

Only uses: situations and reach arrows. Still no operators, one more chapter.

1. From the chalkboard's map, list everything reachable from: s1, s2, s3, s4.
2. Design a reach map for a game with three saves: A = start, B = mid-game, C = final boss beaten. You can load forward through play but there's no way to un-beat the boss. Which arrows exist? Does C reach anything?
3. The choice that matters: in your quiz map, should s1 reach *itself*? Give one reading of "live alternative" where yes makes sense, and one where no does. (Don't resolve it, just see that it's a genuine design decision. Chapter 6 turns this exact decision into mathematics.)

ANSWERS

1. s1: {s2, s3}. s2: {s2}. s3: {s2}. s4: { }, nothing.

2. A→B, B→C, and if "alternatives from here" includes "stay where I am," each save reaches itself; C reaches only itself (or nothing, on the stricter reading), no arrow back from C, because the boss stays beaten. Both versions are defensible *if you say which you chose.*

3. Yes-reading: "the way things actually are is surely one of the ways things could be", the actual counts among the alternatives. No-reading: "alternatives means *other* options, reach is for what could be *instead*." Hold this disagreement carefully; in Chapter 6 it becomes the difference between two famous logics.

CHAPTER 4

The Two New Operators

G gets its upgrade: box, diamond, and a duality you get for free.

PROFESSOR LOGIC

All the ingredients are on the bench: profiles (Chapter 2) and reach (Chapter 3). Now we wire the operators, the G upgrade, and there are exactly two, and you already know what they should mean because you've been saying them all your life:

NECESSARY, written as a box, □, "must, has to, in every live alternative."

POSSIBLE, written as a diamond, ◇, "might, could, in at least one live alternative."

Their definitions write themselves from the ingredients:

THE CHALKBOARD — Box and diamond, defined and computed FORCED

verified: modal_4_1

```
  □P at situation s  =  P is true at EVERY situation s reaches
  ◇P at situation s  =  P is true at AT LEAST ONE situation s reaches

Run it on Chapter 3's quiz map (arrows: s1→s2, s1→s3, s2→s2, s3→s2)
with P = "I pass the quiz", profile:   s1:1  s2:1  s3:0  s4:0

  □P at s1:  P at s2? yes. P at s3? no.  →  □P = 0  (not a must)
  ◇P at s1:  some reached situation with P? s2. → ◇P = 1  (a might)
  □P at s2:  reaches only s2; P there? yes  →  1
  □P at s3:  reaches only s2; P there? yes  →  1   (!!)
  □P at s4:  reaches NOTHING... "P at every reached situation"
             has no cases to fail  →  1, vacuously.
  ◇P at s4:  "some reached situation with P" — none reached → 0

Complete enumeration, situation by situation. The book's proof
technique, unchanged since Chapter 3 of the logic book.
```

SIMPLIFIED FOR CLARITY

□ = "true in **all** the places the arrows go." ◇ = "true in **some** place the arrows go." That's it, must and might are quantifiers over your reach map. Notice everything depends on the arrows: at s3 you *failed* the quiz, yet □P holds there, because every live alternative *from* s3 is a passing one. Box talks about where you can go, not where you are.

PROFESSOR LOGIC · RIGOR CHECK

Two honesty items from that board, because good machines have edge cases and we read them aloud. **One:** s4, the loner reaching nothing, makes □P true *vacuously*. "Every situation I reach" is a promise about zero situations, and a promise about nothing can't be broken. Feels weird; is standard; same logic as "every unicorn in this room is purple." We flag it rather than hide it. **Two:** at s3, □P = 1 while P = 0, "*necessarily P*" *true where P itself is false!* Before you file a complaint: nothing forced s3 to reach itself, so its own failing column never got checked. Whether a situation *should* count among its own alternatives is exactly the design decision from Chapter 3's exercise, and in Chapter 6 it becomes an axiom with a price tag.

THE CHALKBOARD — The duality: one operator free of charge FORCED
verified: modal_4_2

```
Claim:   ◇P  =  NOT □ NOT P      ("might" = "not forced-otherwise")

Read it in English first: "P is possible" says the same thing as
"it is NOT the case that every alternative makes P false."

Check it mechanically on the quiz map at s1:
  NOT P profile:  s1:0  s2:0  s3:1  s4:1
  □(NOT P) at s1: NOT-P at s2? no. → □(NOT P) = 0
  NOT □(NOT P) at s1 = 1   ...and ◇P at s1 = 1. Match.
Checked at every situation of the map: matches at all four.

Just like implication in the logic book (A→B built from NOT
and OR), diamond is NOT a new primitive. Buy box, get diamond
free. Machines this economical are machines you can trust —
fewer parts, fewer places to hide a mistake.
```

TRY IT YOURSELF, CHAPTER 4

Only uses: profiles, reach maps, and the two definitions above. Complete enumeration is the whole method.

1. Same quiz map, new sentence Q = "I studied", profile s1:0, s2:1, s3:0, s4:0. Compute □Q and ◇Q at s1, s2, and s3.
2. Your game-saves map from Chapter 3 (use the self-reaching version: A→A, A→B, B→B, B→C, C→C). Let W = "the boss is beaten", profile A:0, B:0, C:1. Where is ◇W true? Where is □W true? Translate your □W answer into one sentence of gamer English.
3. Duality drill: on the quiz map, compute □(NOT Q) at s1 (use your NOT-Q profile), then NOT-that, then compare with your ◇Q at s1 from question 1. Do they match, as the duality board promises?

ANSWERS

1. At s1 (reaches s2, s3): □Q = 0 (fails at s3), ◇Q = 1 (holds at s2). At s2 (reaches s2): □Q = 1, ◇Q = 1. At s3 (reaches s2): □Q = 1, ◇Q = 1.

2. ◇W: true at B (reaches C) and at C; false at A *if* A reaches only A and B, check your arrows: with A→A, A→B only, ◇W at A = 0. □W: true at C only (C reaches just C, W holds there). Gamer English: "only from the final save is the boss *guaranteed* beaten wherever you load."

3. NOT-Q profile: s1:1, s2:0, s3:1, s4:1. □(NOT Q) at s1: NOT-Q at s2? no → 0. NOT of that = 1. And ◇Q at s1 was 1. Match, as forced.

CHAPTER 5

θ Goes Modal

True here, valid everywhere, and the one law you get before choosing anything.

PROFESSOR LOGIC

Time for the third knob. In the logic book, θ decided which values count as "yes." In calculus, θ became a tolerance. Here, θ splits into **two levels of counting**, and the split is the point:

Level one, true at a situation. "□P at s1" came out 0 on our quiz map. Local. One column's verdict.
Level two, valid on the map. A formula that comes out true at *every* situation, no matter *what* profiles you feed the letters. Not "true here." Unbreakable-on-this-map. *That* is what deserves the name modal **law**.

And here's the question that organizes the rest of the book: **which formulas are valid?** Watch closely, the answer will depend on the *shape of the reach map*. Different arrows, different laws. (Knobs, consequences. Same story since page one of the series.)

THE CHALKBOARD — The free law: K FORCED verified: test_modal_K

```
One law is valid on EVERY reach map — any situations, any arrows,
any profiles. It's called K, and it says box respects implication:

   K:   □(P → Q)  →  (□P → □Q)

English: "if every alternative makes P-leads-to-Q true, and every
alternative makes P true, then every alternative makes Q true."
Why it can't fail: at any situation s, both premises speak about
the SAME reached situations. In each of them, P→Q holds and P
holds — so Q holds (that's the logic book's modus ponens, running
inside each column). No arrow-shape can break that.

Checked by complete enumeration over every reach map on three
situations — all 512 of them, every profile, every situation.
Zero failures. K is the floor: the logic of box BEFORE you've
chosen anything about reach.
```

SIMPLIFIED FOR CLARITY

θ now has two settings: *true-at-a-situation* (local) and *valid* (true at every situation under every profile, a law of the map). Exactly one law comes free with the machinery itself: **K**, "box distributes over implication." It holds on every map that can exist. Everything else, everything, costs an arrow-shape. Next chapter: the price list.

PROFESSOR LOGIC · RIGOR CHECK

Look at what is *not* on the free list, because it's shocking the first time. Is **□P → P** valid, "what's necessary is true"? Sounds like it should be free; sounds like the meaning of the word. It is **not** valid on every map: our quiz map broke it at s3 last chapter (□P = 1, P = 0): the necessity was all "over there," in the situations s3 reaches, and s3 never sees itself. "Necessary implies true" is not part of the machinery, *it's a property some reach maps have and some lack*. If that sentence feels wrong, good: that feeling is a reified intuition, a habit from one map mistaken for a law of all maps, and Chapter 6 is where we turn the habit into an explicit, priced, optional purchase.

TRY IT YOURSELF, CHAPTER 5

Only uses: box/diamond computation and the local-vs-valid distinction. Enumeration on tiny maps.

1. In your own words: what's the difference between "□P is true at s2" and "□P → P is valid on the map"? Which one is a claim about a law?
2. The tiny map: situations a, b; single arrow a→b; profile P: a:0, b:1. Compute □P at a and P at a. What does this pair of values tell you about the formula □P → P on this map?
3. Why can't the same trick break K? Try to describe (no formal proof needed) what you'd need the arrows to do for K to fail at some situation, and why "the same reached situations serve both premises" slams that door.

ANSWERS

1. The first is local, one situation's verdict, with specific profiles. The second says: at every situation, under every profile, the formula holds, unbreakable on this map. Only the second is a law-claim.

2. □P at a: P at b? yes → 1. P at a = 0. So at a, □P → P is 1 → 0 = false: the formula fails at a, hence is *not valid* on this map. (You just built your first countermodel. Keep it; it stars in the next chapter.)

3. To break K you'd need a situation where every reached column satisfies P→Q, every reached column satisfies P, yet some reached column fails Q, but that column would then satisfy P and P→Q and not Q, which no single column can do (that's just modus ponens inside the column). The premises and conclusion quantify over the *same* reached set; there's no gap for the arrows to exploit.

CHAPTER 6

The Ladder

Every famous modal axiom is the price sticker on a shape of reach. Here is the ladder, rung by rung, with receipts.

PROFESSOR LOGIC

Now the chapter this book exists for. Beyond the free law K, modal axioms are **purchased**, and the currency is *constraints on the reach map*. Each constraint you impose on your arrows forces exactly one new law valid. And, this is the half most books skip, **without** the constraint, that law genuinely fails somewhere. Both halves matter. Watch:

THE CHALKBOARD — Rung T: self-reach buys "necessary implies true"

FORCED **verified: test_modal_T**

```
CONSTRAINT (reflexive): every situation reaches ITSELF.
   "the actual counts among its own alternatives"

BUYS:   T:  □P → P     valid on every self-reaching map.
Why: □P at s checks all reached situations — with self-reach, s's
OWN column is one of them. If every reached column has P, s does.

AND THE OTHER HALF — without self-reach, T fails. Witness (your
Chapter-5 countermodel!): a→b only, P: a:0 b:1.
   □P at a = 1 (b checks out), P at a = 0.  T broken at a.
Checked exhaustively: on all 512 three-situation maps, T holds on
every reflexive one AND fails on some non-reflexive one. The
constraint isn't decoration. It is doing the entire job.
```

THE CHALKBOARD — Rung S4: chaining reach buys "necessity is stable"

FORCED **verified: test_modal_4**

```
CONSTRAINT (transitive): if s reaches t and t reaches u, s reaches u.
   "alternatives of my alternatives are my alternatives"

BUYS:   4:  □P → □□P    ("if P is a must, it's a must that it's a must")

WITHOUT it — even keeping self-reach! — 4 fails. Witness:
  a,b,c all self-reaching; a→b, b→c, but NO a→c.  P: a:1 b:1 c:0
  □P at a: checks a,b — both 1 → □P at a = 1
  □□P at a: needs □P at a AND at b; □P at b checks b,c → c fails
            → □P at b = 0 → □□P at a = 0.   4 broken at a.
Reflexive+transitive together = the system named S4.
```

THE CHALKBOARD — Rung S5: mutual reach buys "possibility is stable"

FORCED **verified: test_modal_5**

```
CONSTRAINT (symmetric): if s reaches t, then t reaches s.
   "reach runs both ways"

BUYS (with the rungs below it):   5:  ◇P → □◇P
   ("if P is possible, then it's a must that P is possible")

WITHOUT symmetry — even with self-reach AND chaining — 5 fails:
  a,b self-reaching; a→b; no b→a.   P: a:1 b:0
  ◇P at a: a's column has P → 1
  □◇P at a: needs ◇P at a AND at b; ◇P at b checks only b → 0
            → □◇P at a = 0.   5 broken at a.
Reflexive+transitive+symmetric = S5: reach is an equivalence.
It carves situations into clusters where everyone sees everyone,
both ways: the "view from everywhere." Inside a cluster, no
vantage matters; every situation is modally interchangeable.
The famous logic of "absolute" necessity is exactly the logic of
THAT arrow-shape. No more, no less. (Keep the word "absolute" in
your pocket. Next chapter it gets S5 into trouble.)
```

SIMPLIFIED FOR CLARITY

The ladder: **K** (free) → **T** (self-reach: must implies is) → **S4** (+chaining: must is stably must) → **S5** (+mutual: might is stably might). Every rung = one arrow-shape = one axiom, proved in both directions: the shape forces the law, and the law dies without the shape. Nothing was decreed. Everything was purchased, and you've now seen every receipt.

The man who built the ladder before the ladder had rungs HISTORY

1912–1932. The systems S1 through S5 are older than the reach maps that explain them. **C.I. Lewis**, a Harvard philosopher, was offended, genuinely, productively offended, by a quirk of the logic book's implication: since A→B is just OR(NOT A, B), a *false* A makes A→B true automatically. "The moon is cheese, therefore you're the pope": *true*, classically, since the moon isn't cheese. Lewis wanted an implication with muscle, "A **strictly** implies B" meaning □*(A→B)*: every alternative that makes A true makes B true. To build it he needed a logic of □ itself, and in his 1932 book (with C.H. Langford) he laid out five candidate systems, S1–S5, *axiom lists without a semantics*, ladders with no explanation of what the rungs stood on. For thirty years, which system was "right" was a shouting match. Then Kripke's arrows arrived (Chapter 3), and the shouting match dissolved into a price list: the systems differ by *arrow-shape*, and "which is right?" became "which shape fits your job?", which is next chapter's whole subject.

PROFESSOR LOGIC · RIGOR CHECK

The both-halves discipline deserves one more spotlight before we move on. Showing "reflexive maps satisfy T" alone proves *compatibility*, maybe T held anyway, for free, like K. Showing "T fails on a non-reflexive map" proves the constraint is *load-bearing*. Only together do they justify the word **derived**. This is the exact standard the companion code enforces (the `test_modal_*` stamps run both directions across all 512 three-situation maps), and it's a standard worth exporting far beyond logic: whenever anyone tells you "X causes Y," the question "and does Y fail without X?" is the difference between a mechanism and a coincidence.

TRY IT YOURSELF, CHAPTER 6

Only uses: box/diamond computation on small maps, and the local-vs-valid distinction. Every check is finite and by hand.

1. Verify rung T's positive half on the two-situation map a→a, b→b, a→b with profile P: a:0, b:1. Compute □P and P at both situations; confirm □P → P holds at each.
2. Re-run the S4 countermodel yourself from the chalkboard (a,b,c self-reaching, a→b, b→c, P: 1,1,0): compute □P at a, □P at b, then □□P at a. Confirm the failure.
3. Design question: your game-saves map (A→B→C, all self-reaching, no arrows back). Which of the three constraints, self-reach, chaining, mutual, does it satisfy? So which rungs of the ladder are guaranteed valid on it, and which axiom might fail?

ANSWERS

1. □P at a: checks a (0) → □P = 0, so □P→P is 0→0 = true at a. □P at b: checks b (1) → 1, and P at b = 1, so 1→1 = true. Holds at both, as reflexivity forces.

2. □P at a = 1 (a,b both 1). □P at b = 0 (c fails). □□P at a needs □P at both a and b → 0. So □P→□□P is 1→0: fails. The missing a→c arrow is the hole the failure escaped through.

3. Self-reach: yes. Chaining: yes (A→B→C and A→C, wait, did you include A→C? Transitivity *requires* it; add it or your map isn't transitive!). Mutual: no (nothing returns from C). So T and 4 are guaranteed; the 5 axiom has no guarantee, and indeed ◇(boss-not-beaten) is true at A but not a must-be-possible from C's perspective. A save system is an S4 world, not an S5 one. Time only runs forward; the ladder just told you so.

CHAPTER 7

One Box, Many Jobs

The machinery never told you what □ means. That silence is its superpower, and its price sticker.

PROFESSOR LOGIC

Reread everything we built and notice a strange silence: at no point did the machinery define what □ *means*. We read it "necessary", but the math only ever knew "true at every reached situation." The reading was ours. STIPULATED, always.

And that's not a defect; it's a product line. Swap the reading of □ and the reach relation, keep every gear, and the same machine does a different job. Here's the catalogue, and watch what happens to the ladder in each row, the constraints stop being abstract and start being *claims about the job*:

THE CHALKBOARD — Same machine, four readings STIPULATED readings — FORCED consequences

```
READING of □        REACH means              Is self-reach right?
──────────────────────────────────────────────────────────────────
"known"             situations my            YES: what I KNOW must be
 (knowledge)        information can't        actually true — knowledge
                    rule out                 of falsehoods isn't
                                             knowledge. Keep T.
"obligatory"        situations where         NO: what OUGHT to be
 (duty)             all duties are met       often ISN'T. The world is
                                             not among the ideal
                                             worlds. Drop T!
"always later"      future moments           Chaining yes (later than
 (time)             reachable from now       later is later). Mutual
                                             NO — time doesn't
                                             run back. S4-ish, not S5.
"provable"          (cited, Ch. below)       Shockingly NO — see
 (arithmetic)                                the history box.

One machine. The AXIOMS TRACK THE JOB. Choosing a modal logic
IS choosing which constraints your reading can honestly pay for.
```

SIMPLIFIED FOR CLARITY

□ is a template. Read it as "known" and T is mandatory (knowing implies true). Read it as "ought" and T is *wrong* (ought doesn't imply is, look outside). Read it as "always going to be" and symmetry dies (no arrows back through time). The ladder isn't one true staircase; it's a menu, and the job picks the rungs. **That** is what "the full mechanics" buys you that memorizing one system never could.

The catalogue's engineers HISTORY

1951. G.H. von Wright launches *deontic logic*, □ as "obligatory", and the field immediately confirms the chalkboard: T must go (ought ≠ is), replaced by the gentler D: whatever is obligatory is at least permitted. **1950s. Arthur Prior** builds *temporal logic*, □ as "at all future times," reach as time's arrow, asymmetric by nature, and now the workhorse of verifying that software can't reach a crashed state. **1962. Jaakko Hintikka**'s *epistemic logic*, □ as "known": T stays (knowledge is factive), but S4's "if I know, I know that I know" and S5's "if I don't know, I know that I don't" ignited debates about self-awareness that still run, arguments, note, about *which arrow-shapes fit knowing*, conducted in exactly the vocabulary you now own. **1930s–1970s.** And the strangest room in the catalogue: read □ as **"provable in arithmetic."** Building on Gödel's incompleteness work, **Martin Löb** showed (1955) that "provable implies true", the T axiom itself, *cannot in general be proven inside the system*. The resulting logic, GL, drops T and runs on a reach relation with no self-reach at all. We cite this rather than derive it; it needs machinery beyond this book, but mark what it means: for the provability reading, the most "obvious" axiom on the ladder is the one the mathematics refuses. Obviousness is not a receipt.

PROFESSOR LOGIC · THE ANTI-REIFICATION FINALE

Now the series' standing warning, in its modal costume, and here it has *two* heads. **Head one: don't reify the worlds.** The columns are bookkeeping. The machine runs identically whether "situations" are universes, save files, or rows in a spreadsheet; anyone who tells you modal logic *requires* belief in parallel realities is selling metaphysics with math's price tag stuck on. **Head two, subtler: don't reify the ladder.** After Kripke, it's tempting to take that pocketed word "absolute" out, crown S5 "the real logic of necessity," and treat the rest as damaged goods. But you've now watched T be mandatory for knowledge, false for duty, and *refused by arithmetic itself* for provability. There is no view from nowhere on the ladder. There are jobs, and shapes of reach that fit them, and axioms as the receipts. The question "which modal logic is true?" dissolves, like "which carrier is the true calculus?" before it, into the engineer's question: **which constraints can your reading honestly pay for?** Ask that one. It has answers.

TRY IT YOURSELF, CHAPTER 7

Only uses: the ladder from Chapter 6 plus the readings above. No new machinery; that's the point.

1. For the "known" reading: translate the S4 axiom □P → □□P into an English sentence about knowledge. Does it strike you as true of people? Of a perfect database? (Both answers are defensible, say why.)
2. For the "obligatory" reading: produce one real-world sentence witnessing that □P → P fails, something that ought to be the case and isn't. (Tragically easy. That ease IS the argument against T here.)
3. For the temporal reading (□ = "at every later moment"): which axiom does the sentence "whatever will always hold, will always always hold" express, and which arrow-shape from Chapter 6 pays for it?
4. A classmate says: "S5 is the correct modal logic; the others are approximations." Using one row of the catalogue board, write the two-sentence reply an engineer would give.

ANSWERS

1. "If I know P, then I know that I know P." For humans: doubtful; you know plenty you'd hesitate to claim you know you know. For a database with a query log: plausible; it can check its own contents. The debate is real and 60 years old; you just stated both sides in arrow-vocabulary.

2. Any honest example works: "Everyone ought to be safe crossing the street, and not everyone is." Ought-worlds are reached; the actual world isn't among them. T falls.

3. That's 4: □P → □□P, paid for by chaining (transitivity): a moment later than a later moment is a later moment.

4. "S5's axioms are receipts for mutual reach, but read □ as 'obligatory' and even T fails, let alone 5; read it as 'provable' and arithmetic itself refuses T. So S5 isn't the correct logic; it's the correct logic *for equivalence-shaped reach*, which some jobs have and most don't."

CHAPTER 8

Your Final Challenge

Engineer a modal system, end to end, for a job you choose. Every tool you need is behind you.

PROFESSOR LOGIC

Inventory check. You can spot the must/might layer in ordinary talk and say exactly what {0,1} loses. You can upgrade V to profiles, draw reach, and compute □ and ◇ by honest enumeration, vacuous cases and all. You can split θ into local truth and validity, you know the one free law and *why* it's free, and you hold receipts for every rung of the ladder, both halves, forces-and-fails-without. And you know the machinery's deepest secret: it never chose a meaning for □. You do that. Per job.

So: do it.

THE WHOLE BOOK, IN ONE BREATH

V became profiles across situations. G gained box and diamond, quantifying over reach. θ split into true-here and valid-everywhere. K came free; every other axiom is the receipt for a shape of reach. And the meaning of □ was never in the machine; it's a choice you make, and defend, per job.

YOUR FINAL CHALLENGE, BUILD A MODAL SYSTEM FROM SCRATCH

Uses everything. Nothing new. Show your work at every step.

1. **Pick a job and a reading.** Choose what YOUR □ means. Ideas: □ = "guaranteed by my study plan" · "true on every save file I can still load" · "promised to my team" · "true however the group vote goes." Write the reading in one sentence.
2. **Build V.** Name 3–4 concrete situations for your job. Pick two sentences that matter and write their profiles.
3. **Draw reach.** Which situations are live alternatives from which? Defend each arrow, and each missing arrow, in a phrase.
4. **Compute.** Evaluate □ and ◇ of one of your sentences at every situation, by enumeration. If you have a no-outgoing-arrows situation, handle its vacuous □ out loud like Chapter 4 did.
5. **Interrogate the ladder.** For each constraint, self-reach, chaining, mutual, say whether YOUR reading can honestly pay for it, in one sentence each. Then name your system's rung (or between-rungs position) and which famous axioms are guaranteed valid on your map.
6. **Find your countermodel.** Pick one ladder axiom your map does NOT guarantee and exhibit the failure: the situation, the profile, the computation. (If every axiom holds, your reach is an equivalence, say so, and say whether that's fitting or suspicious for your job.)
7. **Name the price and stay honest.** One sentence: what did your arrow choices cost (what can your system no longer distinguish or express)? One more: what does your map ignore about the real job, and what would it take to reify it anyway?

WHAT A STRONG ANSWER LOOKS LIKE (SKETCH)

Reading: □ = "true on every save I can still load." *Situations:* A (start), B (mid), C (post-boss); arrows A→A, A→B, A→C, B→B, B→C, C→C (self-reach: current save loadable; chaining: loadable-from-loadable is loadable, checked A→C is present; no arrows back: the boss stays beaten). *Compute:* W = "boss beaten" (0,0,1): ◇W everywhere... careful, at C yes, B yes, A yes (A reaches C). □W only at C. Vacuous cases: none (every save self-loads). *Ladder:* self-reach yes, chaining yes, mutual NO, this is an S4 system; T and 4 guaranteed. *Countermodel to 5:* at A, ◇(NOT W) = 1 (A loads A where boss unbeaten), but □◇(NOT W) fails at A because ◇(NOT W) at C = 0. So possible-doesn't-stay-possible: progress destroys options, which is exactly what saves are FOR. *Price:* the system can't express "undo"; anything mutual-shaped is invisible to it. *Honesty:* real play has crashes, cloud restores, and rage-quits my arrows ignore; the map serves the job precisely because it leaves them out, and I'd sooner redraw it than believe it.

PROFESSOR LOGIC

Three machines now: truth, change, and alternatives. Same knobs every time. Same discipline every time, derive it or label it, audit before admiring, both halves of every claim, and never mistake the scaffolding for the sky.

Somewhere ahead of you is a classroom, or a textbook, or a very confident person online, presenting one of these machines as The Truth, no knobs visible, no receipts offered. You know what to ask for now. Ask for it kindly.

But ask for it. Every time. Even here.

A note for teachers and the curious. Boards stamped `test_modal_K`, `test_modal_T`, `test_modal_4`, and `test_modal_5` cite the companion repository (mathoflogic/student, `tests/test_modal.py`), where each ladder claim is checked in both directions, constraint forces axiom; axiom fails without constraint, by complete enumeration over all 512 reach relations on three situations. Boards stamped `modal_N_M` are complete finite computations reproducible by hand as printed. The history is standard: Aristotle's future contingents (De Interpretatione 9); Leibniz's possible worlds (Theodicy, 1710); C.I. Lewis's strict implication and S1–S5 (1918; Lewis & Langford, Symbolic Logic, 1932); Kripke's relational semantics (1959 completeness theorem, published at nineteen; Semantical Considerations, 1963); von Wright's deontic logic (1951); Prior's temporal logic (1950s); Hintikka's epistemic logic (Knowledge and Belief, 1962); Löb's theorem (1955) and the provability logic GL, cited not derived. Łukasiewicz's third value (1918) appears as the companion volume's alternative answer to Aristotle. As Professor Logic insists: check it yourself.